

AI Usage Log

Question 1, Parts 1 and 2

Prepared by	Jeremiah Igbiniedion
AI tool used	ChatGPT
Purpose	Document how AI supported the work, the prompts used, and the corrections made before submission.

Declaration

I used ChatGPT as a support tool for analysis, organisation and drafting. I reviewed the output against the supplied requirements, challenged unsupported assumptions, changed the test case selection, refined the priorities and verified that I could explain every item included in the final documents.

1. Question 1, Part 1

Risk Based Test Plan

What I used AI for

I used ChatGPT to help organise my analysis of the digital wallet transfer limits and account tier requirements into a structured risk based test plan. It helped identify unclear requirements, generate clarification questions, group risks by priority and present the test strategy professionally.

Example prompts

EXAMPLE PROMPT 1

Review the digital wallet transfer limit and account tier requirements. Identify unclear areas, missing business rules, assumptions, dependencies and possible risks. Create clarification questions that should be answered before testing begins.

EXAMPLE PROMPT 2

Using the supplied requirements and identified risks, create a risk based test plan covering the scope, test approach, risk priorities, functional testing, boundary testing, API testing, concurrency testing, scheduled transfers, KYC tier upgrades, pending transfers, reversals and own account exemptions.

What I reviewed and corrected

I compared the generated questions and plan with the original wording. I removed unsupported statements or treated them as clarification questions instead of confirmed behaviour. I also reorganised the risks according to possible financial loss, customer impact and regulatory exposure.

- Confirmed that the Tier 1, Tier 2 and Tier 3 limits were represented accurately.
- Ensured pending, failed and reversed transfer behaviour was included.
- Added coverage for the midnight reset, scheduled transfers and immediate KYC upgrades.
- Treated transfers to the user's own linked bank accounts as exempt from the stated limits.
- Raised the authoritative timezone, partial reversals, transfer type aggregation and scheduled failure handling as clarification points because they were not fully specified.
- Prioritised financial loss, limit bypass, duplicate transfers, race conditions and incorrect allowance calculations.

2. Question 1, Part 2

Top 20 Prioritised Test Cases

What I used AI for

I used ChatGPT to convert the supplied requirements and risk based test plan into structured test cases with IDs, priorities, preconditions, steps and expected results.

Example prompts

EXAMPLE PROMPT 1

Create the top 20 prioritised test cases for the digital wallet transfer limits and account tiers feature. Each test case must include an ID, priority, preconditions, steps and expected result. Focus on quality over quantity.

EXAMPLE PROMPT 2

Select 15 test cases that directly validate the seven supplied requirements. Use the remaining five cases for important high risk scenarios, including concurrency, backend limit enforcement, duplicate submissions, insufficient balance and invalid transfer amounts.

ADDITIONAL DIRECTION

We need to choose the cases that directly validate these exact requirements and hints: Tier 1, Tier 2 and Tier 3 limits, midnight reset, pending transfers, failed and reversed transfers, own linked account exemptions, scheduled transfers executing at 8:00 AM, immediate KYC tier upgrades, and rejection messages showing the remaining daily limit.

What I reviewed and corrected

The initial output gave too much attention to unconfirmed behaviours, including idempotency, partial reversals, shared limits across transfer types and cross day ledger handling. I changed the selection so the final set contained 15 cases that directly validated the stated requirements and 5 cases for major technical and financial risks.

3. Corrections Applied to the Test Cases

- Selected 15 cases with direct traceability to the seven supplied requirements.
- Reserved 5 cases for concurrency, backend enforcement, duplicate submissions, insufficient balance and invalid amounts.
- Added direct boundary validation for Tier 1, Tier 2 and Tier 3.
- Separated the validation of pending, failed and reversed transfers.
- Explicitly verified that scheduled transfers execute at 8:00 AM on the scheduled day.
- Verified that a successful KYC upgrade takes effect immediately without requiring a new login.
- Verified that a rejected transfer displays the correct remaining daily allowance.
- Added backend validation so client side controls cannot be used to bypass transfer limits.
- Added concurrency and repeated submission coverage to reduce the risk of exceeding limits or processing a transfer twice.
- Refined expected results to show the effect on wallet balance, available daily limit and transfer status.

Final review and ownership

I manually checked the final test cases against the requirements, refined the priorities and verified the traceability. I can explain why each case was selected, the data required, the risk it covers and the expected result. The submitted documents are not unreviewed AI output.

Summary

AI supported analysis and drafting. Final scope, prioritisation, corrections and acceptance of the output were completed through my review.

Part Two, API Test Automation

TypeScript, Playwright, Zod and Docker Compose

Candidate	Jeremiah Igbiniedion
AI tool used	OpenAI Codex/ChatGPT
Purpose	Document how AI supported manual API investigation and the conversion of that work into a maintainable automated test suite.

How AI was used

I first performed manual API testing against the pinned local Restful Booker service and inspected responses using curl and controlled exploratory requests. AI then helped convert the verified manual scenarios into structured automation, while also supporting framework design, investigation, documentation and verification.

- Broke the implementation into incremental phases and clear commit boundaries.
- Designed the Playwright API framework, typed client, fixtures, factories, schemas and cleanup helpers.
- Converted verified manual authentication, CRUD, negative, boundary and filtering checks into deterministic automated tests.
- Compared expected and actual API behaviour and helped draft defect reports for confirmed findings.
- Ran linting, strict type checks, test suites, repeatability checks and Git verification.
- Did not commit, push, merge or change branches without my explicit approval.

1. Work Completed With AI Assistance

Environment and framework

- Configured a reproducible Docker Compose service using a pinned Restful Booker image digest, localhost-only access, environment-based port configuration and a /ping health check.
- Added linux/amd64 configuration for Apple Silicon compatibility and documented start, status and stop commands.
- Built reusable Playwright and TypeScript components instead of a flat API script.
- Added environment configuration, RestfulBookerClient, fixtures, booking factories, cleanup tracking, Zod schemas, HTML and JUnit reports, ESLint and strict TypeScript checks.

Automated functional coverage

- Authentication and authorization: valid and invalid credentials, missing and malformed bodies, valid, missing, invalid and stale tokens, plus PUT, PATCH and DELETE protection.
- CRUD and schema validation: create, retrieve, full update, partial update, delete, post-delete 404 verification, persistence checks and a complete lifecycle test.
- Negative and boundary testing: missing fields, blank names, invalid price values and types, Boolean coercion, invalid dates, malformed bodies, long values, unknown fields and injection-style strings.
- Filtering: firstname, lastname, combined names, case sensitivity, check-in, checkout, date ranges, exact boundaries, invalid dates, no-match results, special characters, URL encoding and unsupported pagination parameters.
- Filtering tests retrieve each returned booking ID and validate the booking data instead of treating HTTP 200 alone as proof of correctness.

Defect investigation

AI helped structure the confirmed findings as BUG-API-001 through BUG-API-016, including reproduction steps, expected and actual behaviour, severity, impact, repeatability evidence and links to automated tests. I reviewed the classifications and challenged findings that were not adequately supported.

2. Representative Prompts

PROMPT 1, FRAMEWORK SETUP

Create a reproducible local Restful Booker environment and a minimal TypeScript/Playwright API framework. Use fixtures, helpers, configuration, response schemas, HTML and JUnit reports, and a health test. Keep the target localhost-only. Do not commit or push without my approval.

PROMPT 2, AUTHENTICATION COVERAGE

Implement authentication and authorization coverage for valid credentials, invalid username and password, missing and empty credentials, malformed bodies, valid tokens, missing tokens, invalid tokens, and stale-token behavior. Every protected-endpoint test must create and clean up its own booking.

PROMPT 3, CRUD INVESTIGATION

Investigate POST, PUT, PATCH, and DELETE against a local Restful Booker instance in a separate temporary folder before adding tests to the repository. Use only records created during the investigation, verify persistence, clean everything up, and report confirmed bugs separately from product questions.

PROMPT 4, NEGATIVE AND FILTERING COVERAGE

Test wrong types, malformed bodies, price boundaries, invalid dates, injection-style strings, name filters, date-range filters, case sensitivity, pagination behavior, and special-character encoding. Verify filtering correctness by retrieving the returned booking IDs, not just by checking for HTTP 200.

3. Human Corrections and Decisions

Configuration and Git safety

- Required explicit approval before every commit or push and reviewed each proposed scope and commit message.
- Corrected hard-coded admin credentials in the fixture so username and password came from environment configuration and could be overridden in CI or other environments.

Evidence-based defect classification

- Did not classify duplicate-date bookings as a confirmed defect because the API exposes no room, property, capacity or resource identifier. Added a characterization test and documented the overbooking risk as a product question.
- Kept unauthenticated GET /booking access as an observation because the documented test API may intentionally allow public reads.
- Used Playwright expected failures only for confirmed defects. Product questions and limitations were not marked with test.fail.
- Limited token-expiry claims to the available evidence: fabricated stale-token rejection, restart invalidation, no deterministic natural expiry and no documented expiration mechanism.
- Retained case-sensitive filtering as a usability or product question because the documentation does not specify case-insensitive matching.

Wire-level request correction

Exploratory JavaScript fetch and Playwright initially produced different results for empty bodies. I required further investigation with `curl --data-binary ''`. The client was corrected to send raw bodies using a Buffer, ensuring a genuinely zero-byte request. The resulting POST, PUT and PATCH automation then matched the observed wire behaviour.

4. Confirmed Findings Supported by Automation

- BUG-API-001, Invalid credentials return HTTP 200.
- BUG-API-002, Tokens have no expiration or revocation mechanism.
- BUG-API-003, GET /ping returns HTTP 201.
- BUG-API-004, Missing or empty credentials receive generic authentication errors.
- BUG-API-005, Negative booking prices are accepted.
- BUG-API-006, Blank and whitespace-only customer names are accepted.
- BUG-API-007, Missing mandatory booking fields produce HTTP 500.
- BUG-API-008, Non-boolean deposit values are accepted and coerced.
- BUG-API-009, Invalid date ordering and zero-night stays are accepted.
- BUG-API-010, Operations on nonexistent bookings return HTTP 405.
- BUG-API-011, Booking field types are inconsistently validated.
- BUG-API-012, Price coercion and truncation cause data corruption.
- BUG-API-013, Impossible calendar dates are silently normalized.
- BUG-API-014, Empty and array POST bodies cause HTTP 500.
- BUG-API-015, Check-in filtering excludes the documented lower boundary.
- BUG-API-016, Invalid filtering dates produce HTTP 500.

Final review and ownership

I manually executed and inspected API requests, reviewed the AI-assisted automation, corrected unsupported assumptions and implementation details, and verified the final suite through linting, type checks, repeated test runs and result inspection. I can explain the framework, each test, each confirmed defect and the corrections made. The submitted work is not unreviewed AI output.

Part 5, CI/CD

AI usage supplement, appended without changing the previous nine pages

AI tool	OpenAI Codex / ChatGPT
Purpose	Support CI/CD planning, implementation, review, debugging, documentation and verification.
Final status	Pipeline completed and GitHub Pages dashboard live

How AI was used

- Reviewed the Part 5 requirements and divided implementation into progressive milestones.
- Designed the lint and type-check gate, four-shard matrix and pinned Docker service.
- Prepared Playwright for test-level sharding and designed known-bug expected-failure handling.
- Implemented artifact uploads, blob-report merging, combined HTML and JUnit reports, and the Pages dashboard.
- Reviewed secret handling, workflow YAML, local verification results and real Actions runs.
- Helped identify and correct security, sharding and reporting issues.

Representative prompts

PROMPT 1, PIPELINE DESIGN

Create a plan for Part 5 that runs lint before the complete Part 2 Playwright API suite, starts Restful Booker as a GitHub Actions service, runs jobs in parallel where possible, uploads reports and failure artifacts, handles expected-failure tests without keeping CI red, and includes nightly execution and test sharding.

PROMPT 2, REPORTING AND PAGES

Replace the separate shard HTML reports with one combined Playwright report, improve the final HTML presentation, add a professional dashboard with recent workflow history, and prepare it for GitHub Pages.

Human direction

- Required credentials to come from environment variables and GitHub Secrets.
- Required explicit approval before commits or pushes.
- Selected one merged report instead of four shard reports.
- Selected successful main runs only for Pages deployment.
- Selected a professional light dashboard with recent run history and generic Restful Booker identity.

Corrections made to AI-assisted output

Shard distribution

Changed file-level sharding from 27, 53, 0 and 21 tests to test-level distribution of 26, 25, 25 and 25.

Credential security

Removed duplicated hardcoded username and password values and required configuration through environment variables and secrets.

Report usability

Replaced four independent shard reports with merged blob reporting and one combined HTML and JUnit result.

Pages privacy

Excluded error contexts, JUnit XML, ZIP archives, traces, screenshots and other diagnostics from the public bundle.

Deployment safety

Restricted deployment to successful main runs so branches, pull requests and failed runs cannot overwrite the public green report.

Human verification and ownership

- Reviewed workflow triggers, dependencies, service configuration, credentials, sharding, expected failures and artifact handling.
- Ran and checked linting, TypeScript, Playwright, shard distribution and report merging.
- Reviewed feature and develop Actions runs, artifact inventories, JUnit totals and dashboard layouts.
- Confirmed that the GitHub Pages dashboard is live at <https://dionjerry.github.io/quality-engineering-project/>.
- Approved the implementation decisions and can explain or modify the pipeline during the live defence.

Final responsibility statement

AI supported planning, implementation, debugging, documentation and verification. I selected the architecture and deployment policies, corrected the credential, sharding, reporting and privacy issues, supplied repository secrets, approved Git operations and verified the resulting behaviour. The submitted work is reviewed and understood, not unexamined AI output.

AI USAGE AUDIT

Task A: AI Test Design with Critique
Digital Wallet Transfer Limits & Account Tiers

PREPARED BY	AI TOOL	DOCUMENT TYPE
Jeremiah Igbinedion	ChatGPT	AI Usage Record

76 GENERATED | HUMAN-LED CRITIQUE | 38 FOCUSED CASES

1. Purpose of This Audit

This record documents exactly how ChatGPT was used for Task A, what was generated, what I reviewed manually, which follow-up prompts I supplied, and how the final output changed. It is intended to provide transparent evidence of AI assistance without presenting AI decisions as my own analysis.

OWNERSHIP STATEMENT

I manually reviewed the generated suite first and identified the principal duplicates, unsupported cases and business-logic errors. AI was subsequently used to inspect the remaining cases, organise classifications and format the critique.

2. AI Tool and Scope

Item	Details
Tool	ChatGPT
Primary use	Initial comprehensive test-case generation
Secondary use	Additional audit of cases remaining after my review
Documentation use	Organising the AI Critique and producing the final PDF
Human responsibility	Requirement interpretation, business reasoning and final keep, correct, merge or remove decisions

3. Interaction Timeline

Stage	Action	Owner
1	Submitted the complete feature specification and requested extensive test cases.	Me
2	Returned an incorrect earlier PDF limited to 20 test cases.	AI
3	Challenged the artificial limit and requested all cases.	Me
4	Generated a comprehensive suite of 76 cases.	AI
5	Reviewed the suite and identified duplicates, hallucinations and wrong expected results.	Me
6	Inspected the remaining cases and organised the combined classification.	AI-assisted
7	Approved the reasoning and requested the final critique and usage documents.	Me

4. Actual Prompts Used

Prompt 1: Initial comprehensive generation

Part 1: Test Strategy & Spec Analysis (COMPULSORY)

Below is a real-style feature specification. It contains genuine ambiguities, contradictions, and unhandled edge cases; finding them is part of the task.

Feature spec: Digital Wallet Transfer Limits & Account Tiers

A mobile wallet app lets users send money to other wallet users and to bank accounts. Every account belongs to one of three tiers, based on the level of identity verification (KYC) completed.

1. Tier 1 accounts may send up to 20,000 per transaction and 50,000 per day. Tier 2: 200,000 per transaction, 500,000 per day. Tier 3: no per-transaction limit, 5,000,000 per day.
2. The daily limit resets at midnight.
3. Pending transfers count toward the daily limit. If a transfer fails or is reversed, the amount is returned to the user's available daily limit.
4. Transfers to a user's own linked bank accounts are exempt from all limits.
5. Users can schedule transfers for a future date; scheduled transfers execute at 8:00 AM on the scheduled day.
6. When a user completes the next KYC level, the tier upgrade takes effect immediately.
7. If a transfer would exceed the remaining daily limit, it is rejected with a clear error message showing the remaining limit.

Dully and extensivle Generate Test Cases for this SPEC and send a PDF

Prompt 2: Correction after the output was limited

Where did I ask you to create only 20? In this message I never asked you to create only 20. Create all test cases that are in this specification and create the PDF again.

Prompt 3: My manual critique of the 76-case suite

Now based on this document I have some issues with your doc. TC_005 remove, it is repeating; TC_006 is repeating; TC_011 and TC_012 are repeating; TC_021 repeats TC_020; TC_024 repeats TC_023 and TC_022.

TC_029 and TC_030 are wrong because the specification acceptance criteria never states so.

TC_032 is not technically correct. If I made a transfer and my limit is 50,000 and my remaining limit is now 30,000, and it fails the next day after I already made a transfer and my remaining limit is 40,000, the failed transaction from yesterday does not affect today's limit. It remains 40,000. Failed transaction restoration only counts for the same day.

TC_041 repeats TC_036. TC_035 should span across all tiers. TC_044 repeats TC_042 and TC_043. TC_045 is redundant because a scheduled transfer is based on its date and day. TC_046 repeats TC_042.

TC_047 is not correct. You cannot book a future limit in advance. Limits apply on the scheduled date and if it does not work, it fails with an error.

TC_049 repeats TC_048. The specification never says a user can cancel or edit a scheduled transfer, so TC_050 and TC_051 are unsupported. TC_053 repeats TC_052. TC_057 repeats TC_054, TC_055 and TC_056.

TC_059 assumes the transfer and upgrade happen at the same split second. Realistically, transactions for a user should be processed like a queue. TC_060 is unsupported because the specification never discusses downgrade. TC_066, TC_067, TC_069 and TC_070 are repeating. TC_072, TC_073, TC_074, TC_075 and TC_076 are not specified and were hallucinated.

Some tests should now be removed. Look into the remaining tests and identify anything else hallucinated, repeating or incorrect. I am answering based on Task A: AI test design with critique, which requires which generated cases were kept, wrong, redundant or hallucinated and why, where AI helps versus harms test design, and the prompts used.

Prompt 4: Requested audit format

No, do not create another PDF for me. Just send me the other ones that are duplicated, along with mine, and send me the remaining cases. You can send it here so I can review it.

Prompt 5: Create the critique

Based on this, let us create the AI Critique document.

Prompt 6: Produce the critique PDF

Create the PDF now using the same design style.

PROMPT ACCURACY

The detailed critique in Prompt 3 originated from my own review of the comprehensive test suite. It was not a prompt asking AI to invent the critique from scratch.

5. Initial AI Output

The AI produced 76 test cases covering tier limits, daily boundaries, midnight reset, pending and reversed transfers, own-account exemptions, scheduling, KYC upgrades, remaining-limit messaging, concurrency, API, security and data integrity. The breadth was useful, but the suite contained repeated checks and scenarios that could not be traced to the seven stated requirements.

6. My Manual Review and Corrections

I compared the cases against the specification and supplied concrete corrections rather than accepting the generated results.

Review area	My finding
Duplication	I identified repeated limit, midnight reset, own-account, scheduling and KYC cases, including TC_005, TC_006, TC_011, TC_012, TC_021, TC_024, TC_041, TC_044, TC_046, TC_049, TC_053 and TC_057.
Cross-day accounting	I corrected TC_032: a Day 1 failure arriving on Day 2 must not increase the user's Day 2 allowance. Restoration applies only to the daily period where the amount was counted.
Own-account exemption	I required TC_035 to cover all three tiers because the exemption applies to all limits.
Scheduled transfers	I rejected advance limit reservation. Limits must be evaluated on the scheduled execution date. If allowance is insufficient, execution fails with the applicable error.
Invented features	I rejected cancellation, editing, KYC downgrade and other behaviours absent from the specification.
Unrealistic timing	I challenged the assumption that a KYC upgrade and transfer occur at the exact same instant and noted that user transactions should be processed sequentially.
Scope control	I rejected generic infrastructure and security cases that the acceptance criteria did not specify.

7. How AI Was Used After My Review

- Checked the cases I had not explicitly classified and suggested additional duplicates or unsupported assumptions.
- Combined my findings with its secondary review into retained, corrected, redundant, merged and hallucinated groups.
- Helped express the corrected cross-day, own-account, scheduling and KYC cases consistently.
- Structured the final AI Critique document and applied the agreed report design.

IMPORTANT DISTINCTION

AI supported the second-pass audit and documentation. The central business critique, including the cross-day correction and rejection of unsupported scheduling behaviour, came from my manual QA analysis.

8. Outcome of the Audit

Classification	Count
Initial AI-generated cases	76
Retained without major correction	32
Retained after correction or expansion	6
Removed or merged	38
Final focused suite	38

9. Where AI Helped

Benefit	Evidence
Speed	Generated a broad first draft much faster than writing every scenario from a blank page.
Coverage discovery	Identified useful boundaries, midnight transitions, pending transfers, reversals, scheduling and immediate upgrades.
Structure	Produced consistent test IDs, priorities, preconditions, steps and expected results.
Review stimulus	Incorrect or speculative cases still exposed areas that required deliberate specification analysis.
Documentation	Helped classify decisions and turn the review into a readable critique.

10. Where AI Harmed Test Design

Risk	Evidence
False completeness	The high count of 76 included duplicated coverage and therefore overstated the suite's value.
Hallucination	AI introduced cancellation, editing, tier downgrade, callbacks, rollback, logging and other unsupported behaviours.
Wrong expected result	Cross-day restoration could have affected the wrong day's available allowance.
Assumption leakage	Possible technical designs were written as confirmed product requirements.
Scope dilution	Generic security and reliability scenarios distracted from traceability to the supplied specification.

11. Final Assessment

AI was valuable as an accelerator and brainstorming assistant, but the first output was not submission-ready. The suite only became defensible after I traced the cases to the specification, removed duplication, rejected invented features and corrected business logic.

The most important lesson is that a larger AI-generated suite is not necessarily a better suite. Test quality depends on traceability, meaningful differentiation, correct expected results and an understanding of the business rule being validated.

FINAL POSITION

AI can propose, expand and organise test ideas. The QA engineer must remain accountable for scope, interpretation, prioritisation and every expected result submitted.

12. Review Declaration

I reviewed the AI-generated test cases against the supplied specification, identified the principal defects in the generated suite, supplied the business corrections and made the final decisions about which cases should be kept, corrected, merged or removed. I can explain the prompts, classifications and reasoning during a live review.

Prepared by	Task	Status
Jeremiah Igbiniedion	Task A, AI Test Design with Critique	Final

Part 6 Task B: Local AI Test Case Generator

Assessment-wide AI usage supplement

Candidate	Jeremiah Igbinedion
AI tools	OpenAI Codex / ChatGPT
Purpose	Support architecture, implementation, tests, interface design and documentation.
Human control	All Git operations require explicit approval; no live provider call was made without a supplied key.

How AI was used

- Converted the agreed Task B product decisions into a local TypeScript implementation.
- Implemented the exact user-supplied system prompt as a protected constant and unit test.
- Designed strict structured output, deterministic validation and one-sheet Excel export.
- Created a local responsive interface and security controls for environment and runtime credentials.
- Added mocked tests, dependency checks, browser verification, live OpenRouter verification and documentation.

Representative Prompts and Human Direction

Prompt 1 - Product direction

Build a local AI test-case generator using OpenRouter. The user pastes a feature specification and receives an Excel sheet of test cases. Use environment variables for the API key and model.

Prompt 2 - Simplification

Do not create Summary, Requirement Analysis, Clarification Questions, Traceability Matrix, or Quality Audit workbook sheets. I paste the feature specification and it gives me test cases.

Prompt 3 - Exact guidance

Use the user-supplied senior QA system instruction verbatim. Require structured JSON so the backend can validate it before creating Excel.

Human decisions

- Removed GitHub Pages and external hosting from scope.
- Reduced the workbook to one Test Cases worksheet.
- Required pasted text only; no document upload.
- Required ephemeral runtime API-key/model overrides while keeping environment values hidden.
- Required all generated XLSX, CSV and report files to remain ignored.

Corrections, Verification and Ownership

- Rejected an initial multi-sheet workbook design as unnecessarily complicated.
- Preserved the exact approved prompt rather than referencing or paraphrasing it.
- Changed the dependency set after npm exposed an incompatible test-runner resolution.
- Upgraded the test runner and overrode an affected transitive UUID dependency; the final production audit reports zero vulnerabilities.
- Corrected lint, strict typing and duplicate-test assertions until all checks passed.
- Verified desktop and mobile layouts, rejection feedback, mocked paths, and a live nine-case Excel download using the supplied wallet specification.

Trust boundary

The AI response is never trusted directly. It must satisfy the JSON schema and deterministic validation. No workbook is generated when any final case is incomplete, vague, duplicated, unsupported, blocked, or untraceable. Human QA approval remains mandatory.

Review declaration

I reviewed the architecture, prompt, interface, validation rules, tests and generated documentation. I can explain how configuration precedence, OpenRouter structured output, rejection handling and workbook generation work, and I remain responsible for the submitted implementation.